Technical Summary: Tree of Problems: Improving Structured Problem Solving with Compositionality

Kai-Yu Lu

1 Research Problem and Motivation

Large Language Models (LLMs) are often prompted to solve complex reasoning problems using In-Context Learning (ICL), where a few demonstrations inside a prompt guide the model toward the intended solution pattern. Chain-of-Thought prompting (CoT) improves multi-step reasoning by eliciting intermediate reasoning steps, but the paper reports accuracy degradation as problem length increases and limited out-of-distribution (OOD) generalization when the test instance structure differs from the prompt demonstrations. Search-style prompting frameworks such as Tree of Thoughts (ToT) and Graph of Thoughts (GoT) address complex reasoning by exploring multiple intermediate states and aggregating candidates, but the paper characterizes these approaches as unnecessarily complex for tasks where a complex instance is compositional and decomposes into multiple similar subinstances. The motivating gap is a prompting method that directly exploits compositionality to reduce long-horizon error accumulation without requiring search, scoring, or refinement.

2 Related Work

Chain-of-Thought prompting (CoT). CoT elicits step-by-step reasoning and improves performance on reasoning benchmarks. CoT with Self-Consistency (CoT-SC) samples multiple reasoning traces and aggregates answers. The paper describes limited OOD gains from CoT-SC relative to the targeted compositional setting.

Decomposition-based prompting. Least-to-Most (L2M) prompting decomposes a problem into a sequence of subproblems and solves them progressively. Related decomposition strategies aim to reduce complexity by staging the solution process. The present approach focuses on decomposition into multiple analogous subinstances that reuse the same solving pattern and support systematic recombination.

Search-based structured reasoning. ToT treats reasoning as search over a tree of intermediate states, and GoT generalizes this to graph structures with aggregation (and optional refining) of intermediate results. The paper contrasts ToP with these by avoiding search and defining each node as an explicit subproblem instance, with bottom-up composition to produce the final answer.

3 Dataset Construction

The study evaluates prompting methods on benchmark-style task suites and toy symbolic tasks. No parameter training is described; the setup is inference-time prompting.

Table 1: Datasets and construction details as described in the paper.

Dataset / Task Set	Size	Source	Construction, labels, preprocessing, and splits
Sorting (GoT task)	100 examples	GoT evaluation tasks	Input is a list of 32 integers in $[0, 9]$; output is the sorted list. Preprocessing: Not specified in the paper. Split: Not specified in the paper.
Set Intersection (GoT task)	100 examples	GoT evaluation tasks	Input is two sets of 32 elements each; output is the intersection set. Preprocessing: Not specified in the paper. Split: Not specified in the paper.
Keyword Counting (GoT task)	100 examples	GoT evaluation tasks	Input is a text; output is country mentions and counts. Preprocessing: Not specified in the paper. Split: Not specified in the paper.
Last Letter Concatenation (toy)	500 examples	CoT toy tasks	Input is a list of names; output concatenates the last letter of each name. Evaluation varies list length (4, 8, 16). Preprocessing: Not specified in the paper. Split: Not specified in the paper.
Coin Flip (toy)	500 examples	CoT toy tasks	Input describes a sequence of people who flip or do not flip a coin; output is the final coin state. Evaluation varies sequence length (4, 8, 16). Preprocessing: Not specified in the paper. Split: Not specified in the paper.
BBH subset	Not specified in the paper	BIG-Bench Hard (BBH)	Eight tasks: Boolean Expressions, Hyperbaton, Multi-Step Arithmetic Two, Navigate, Object Counting, Tracking Shuffled Objects (3/5/7), Web of Lies, Word Sorting. Preprocessing: Not specified in the paper. Split: Not specified in the paper.

Table 2: Models and experimental settings.

Item	Specification
Language models	`gpt-3.5-turbo` and `gpt-3.5-turbo-instruct`. Additional LLaMA analyses are reported in appendices.
Prompt sources for solving	BBH: CoT prompts from prior BBH work with minor changes. Toy tasks: prompts inspired by prior CoT toy-task prompts. GoT tasks: prompts follow the GoT evaluation setup.
Training	Not specified in the paper (prompting-based evaluation; no parameter training described).
Evaluation split and randomness control	Not specified in the paper.
Hardware, runtime, and cost	Not specified in the paper.
Code availability	Public repository is provided in the paper.

4 Query Protocol and Task Definitions

Task Classes and Assumptions

The paper distinguishes two task classes.

- **Canonical tasks.** A complex instance is divisible into independent subinstances that are similar in form to the original instance, enabling divide-and-conquer composition.
- **Sequential tasks.** A complex instance consists of ordered processing steps where later steps depend on intermediate states produced by earlier steps.

Table 3: Query protocol and task definitions used for evaluation.

Item	Definition
Input	A task instance in natural language or structured text, optionally with a few-shot prompt.
Output	A final answer in the task-required format (for example a sorted list, an intersection set, a coordinate pair, or a final discrete state).
Success standard	Accuracy based on exact-match evaluation of the final answer. For string outputs such as Last Letter Concatenation, correctness requires an exact string match.
Evaluation metric	Accuracy, defined as the fraction of instances where the final predicted answer exactly matches the reference answer.

5 Modeling Approach

Tree of Problems (ToP): Definition and Pipeline

Tree of Problems (ToP) is a compositional prompting framework that constructs and solves a tree where each node represents an explicit subproblem instance related to the root problem. The method comprises three components.

- **Decomposer.** Produces a problem tree by dividing an instance into smaller related instances. Decomposition is algorithmic or performed by prompting with a divide prompt. The tree is parameterized by **breadth** b (number of children per internal node) and **depth** d (granularity of atomic subproblems). The configuration is denoted ToP(b, d).
- **Solver.** Solves leaf nodes using an LLM with a task-specific solve prompt (commonly CoT-style reasoning).
- **Merger.** Combines solved children to produce the parent solution using a merge prompt in a bottom-up manner.

The paper states that, excluding decomposition overhead, the number of inference calls equals the number of nodes in the ToP tree.

Sequential Tasks via Breadth-One Composition

For sequential tasks, the paper uses breadth $b = 1$, creating a chain of grouped steps. Each node applies a subset of processing steps to an intermediate state produced by the previous node. In this setting, explicit merging is not required; each node output becomes the next node input.

Core-Only Formula

The paper provides an explicit step grouping for sequential tasks.

$$G_1 = (p_1, \ldots, p_{\lceil m/k \rceil}),\ \ldots,\ G_k = (p_{m-\lfloor m/k \rfloor+1}, \ldots, p_m). \tag{1}$$

Purpose. Partitions m ordered steps into k consecutive groups to form the ToP$(1, k)$ sequential decomposition used at inference.
Symbols. $(p_1, \ldots, p_m)$ are the ordered processing steps; m is the number of steps; k is the number of groups (corresponding to the ToP depth setting for breadth one); G_i is the i-th consecutive group; $\lceil \cdot \rceil$ and $\lfloor \cdot \rfloor$ are ceiling and floor operators.
Intuition. Long sequences are replaced by shorter segments, reducing long-horizon state-tracking errors.

Usage. The solver is applied sequentially to (s_0, G_1) to obtain s_1, then to (s_1, G_2), continuing until the final state is produced after G_k, matching the sequential-task ToP workflow described in the paper.

Minimal Example (Canonical Task)

For Last Letter Concatenation with four names, ToP$(2, 1)$ splits the list into two sublists of two names each. The solver computes the last-letter concatenation for each sublist. The merger concatenates the two leaf outputs to produce the final string. This example illustrates the core mechanism: identical solving behavior is reused on smaller instances, and composition reconstructs the root solution.

6 Empirical Results

Experimental Setup, Prompts, and Models

The paper evaluates `gpt-3.5-turbo` and `gpt-3.5-turbo-instruct`. BBH prompts follow CoT prompts from prior BBH work with minor changes. Toy-task prompts are inspired by prior CoT toy-task prompts. Hyperparameters such as temperature, top-k, batch size, optimizer, and training steps are not specified in the paper. Hardware and wall-clock cost are not specified in the paper.

Canonical Structured Tasks (GoT Tasks)

ToP improves over GoT on Sorting (0.68 versus 0.28, +0.40) and Set Intersection (0.65 versus 0.46, +0.19), consistent with the compositional structure of these tasks. The improvement on Keyword Counting is smaller (0.31 versus 0.26, +0.05), indicating that decomposition does not fully resolve errors for text-based identification and counting.

Toy Symbolic Reasoning: Length Scaling and Call-Matched Comparisons

Accuracy decreases with longer lists for both CoT and ToP. ToP remains higher at all evaluated lengths, with the largest absolute gap at length 16 (0.444 versus 0.252, +0.192). Under inference-call matching, ToP (match) exceeds L2M for length 16 (0.858 versus 0.742, +0.116) and remains above L2M at lengths 4 and 8. CoT-SC remains substantially lower at length 16 (0.116), indicating that sampling multiple CoT traces does not offset long-chain error accumulation under this task.

BBH Tasks: Gains and Regressions

ToP yields gains on Hyperbaton (0.902 versus 0.804, +0.098) and Word Sorting (0.831 versus 0.619, +0.212). ToP is lower than CoT on Boolean Expressions (0.896 versus 0.924, -0.028), Multi-Step Arithmetic Two (0.736 versus 0.780, -0.044), and Object Counting (0.892 versus 0.928, -0.036). These regressions reflect the limitation described in the paper that reasoning consistency across closely related formulations constrains bottom-up composition for specific tasks.

Sequential Tasks: Length Robustness

ToP improves over CoT on Navigate (0.884 versus 0.864, +0.020) and Web of Lies (0.972 versus 0.920, +0.052). For Coin Flip, ToP remains at 0.998 for 8 people while CoT drops to 0.840, and ToP remains higher at 16 people (0.756 versus 0.718, +0.038). For Tracking Shuffled Objects, ToP exceeds CoT at 5 and 7 swaps (0.440 versus 0.324, and 0.118 versus 0.044), while ToP is slightly lower at 3 swaps (0.524 versus 0.536). These outcomes show improved robustness under increasing sequence length for several tasks.

Table 4: Accuracy on GoT tasks with `gpt-3.5-turbo`.

Task	CoT	ToT (best)	GoT (best)	ToP
Sorting	0.02	0.17	0.28	**0.68**
Set Intersection	0.07	0.25	0.46	**0.65**
Keyword Counting	0.00	0.00	0.26	**0.31**

Efficiency, Cost Proxy, and Reported Discussion

The paper uses the number of inference calls as a cost proxy. For canonical ToP, the number of calls (excluding decomposition) equals the number of nodes in the ToP tree. Hardware usage, wall-clock time, and monetary cost are not specified in the paper.

Limitations Observed in Results

ToP is not uniformly superior across tasks. Lower accuracy relative to CoT on Boolean Expressions, Multi-Step Arithmetic Two, and Object Counting indicates task-dependent failure modes tied to reasoning consistency across closely related subinstances, consistent with the paper discussion.

Table 5: Accuracy on Last Letter Concatenation and comparisons (`gpt-3.5-turbo-instruct`).

List length	CoT	ToP	Improvement (ToP−CoT)
4	0.900	**0.990**	+0.090
8	0.662	**0.854**	+0.192
16	0.252	**0.444**	+0.192

List length	CoT-SC	L2M	ToP	ToP (match)
4	0.908	0.988	0.990	**0.990**
8	0.574	0.870	0.854	**0.932**
16	0.116	0.742	0.444	**0.858**

Table 6: Accuracy on selected BBH tasks with `gpt-3.5-turbo-instruct`.

Task	IO	CoT	ToP
Boolean Expressions	0.908	**0.924**	0.896
Hyperbaton	0.528	0.804	**0.902**
Multi-Step Arithmetic Two	0.032	**0.780**	0.736
Object Counting	0.412	**0.928**	0.892
Word Sorting	0.837	0.619	**0.831**

Table 7: Accuracy on sequential tasks with `gpt-3.5-turbo-instruct`.

Task	IO	CoT	ToP
Coin Flip (4 people)	0.512	**0.998**	**0.998**
Coin Flip (8 people)	0.502	0.840	**0.998**
Coin Flip (16 people)	0.476	0.718	**0.756**
Navigate	0.204	0.864	**0.884**
Tracking Shuffled Objects (3)	0.004	**0.536**	0.524
Tracking Shuffled Objects (5)	0.004	0.324	**0.440**
Tracking Shuffled Objects (7)	0.000	0.044	**0.118**
Web of Lies	0.528	0.920	**0.972**

7 Summary

Tree of Problems (ToP) is a compositional prompting framework for structured problem solving. The method decomposes a complex instance into a tree of explicit subproblem instances parameterized by breadth and depth, solves leaf subproblems using a shared solve prompt, and merges solutions bottom-up using a merge prompt. For sequential tasks, ToP uses a breadth-one decomposition that passes intermediate states across grouped step segments, supported by an explicit grouping formulation.

Empirically, ToP outperforms ToT and GoT on canonical structured tasks, including Sorting (0.68 versus 0.28 against GoT) and Set Intersection (0.65 versus 0.46 against GoT). On Last Letter Concatenation, ToP exceeds CoT as list length increases, and the call-matched ToP variant exceeds L2M at length 16 (0.858 versus 0.742). On BBH, ToP improves performance on Hyperbaton and Word Sorting but underperforms CoT on Boolean Expressions, Multi-Step Arithmetic Two, and Object Counting, reflecting the limitation stated in the paper that reasoning consistency constrains compositional decomposition for specific tasks.